VCU College of Engineering

CS 25-322 AI-generated planning insights powered by Clickstream data

Final Design Report

Prepared for

Mahesh Nair / Tyler Jordan / Emily Croxall

Capital One

By

Priya Choudhary, Bindi Patel, Carissa Trieu, Ivan Emdee

Under the supervision of

Thomas Gyeera

5/9/2025

Executive Summary

The purpose of this project is to collect and analyze real-time clickstream data to provide AI-driven insights for optimizing roadmap planning. Analysis of user interactions on a web application similar to the Capital One Help Center page will allow stakeholders to make better informed decisions with an emphasis on metrics like Average Handle Time and user engagement.

The primary objectives and deliverables of the project include the creation of an intuitive user interface, analysis of clickstream, and implementation of AI/ML models to offer actionable recommendations. A scalable database will also be utilized to handle clickstream data. Using the recommendations, epics, and stories will be automatically generated in JIRA for efficient decision-making.

The final project deliverable will be a fully functional prototype, ready for demonstration to Capital One. Academic deliverables include a project proposal, a preliminary design report, a fall semester poster and presentation by November 2024, a final design report, and a final EXPO poster/ presentation by Spring 2025. This project is relevant for Capital One's Agent Servicing team, as it will allow stakeholders to gain a better understanding of user behavior, improve platform efficiency, and support long-term strategic planning. Ultimately, this project aims to both enhance user experience and operational efficiency.

Table of Contents

Section A. Problem Statement

In today's fast-paced digital world, platform stakeholders need to make informed, data-driven decisions to enhance user experiences, optimize application performance, and plan for future growth. Clickstream data, which captures user interactions within an application, provides valuable insights into user behavior, but many organizations struggle to fully leverage this information to guide their product roadmaps.

Clickstream is a sequence of pages to describe the history of a user's session. This type of data has proven useful in web usage mining and in generating real-time predictions (Bucklin & Sismeiro, 2009). Currently, existing solutions often fall short when it comes to processing clickstream data at scale because it is done by hand. This takes a significant amount of time to do. Delivering AI-powered recommendations, and integrating seamlessly with tools like JIRA for project management will help shorten the time taken. This project aims to bridge that gap by developing a web application that not only captures clickstream data but also uses AI to analyze it and provide actionable recommendations for improving platform features and reducing inefficiencies.

The system will address key stakeholder questions, such as identifying areas to reduce Average Handle Time (AHT), pinpointing where users spend the most time in the application, and highlighting critical features. Additionally, it will integrate with JIRA to automatically generate mock Epics and stories based on accepted recommendations, helping streamline the implementation process. By creating this solution, the project equips stakeholders with a powerful tool to turn clickstream data into meaningful insights, allowing for smarter, data-driven decisions that support strategic roadmap planning.

Our project client is Capital One, they have tasked us with this project because they want to see if it is more efficient to have an AI look over their clickstream data. Rather than having a human look over it. After doing much research, we believe that it will be and hope to prove it with our prototype. The articles we read said that it was faster and more accurate to have an AI look over the data.

The general field of study that the project falls under is data analytics and artificial intelligence, specifically clickstream analysis and artificial intelligence-driven product roadmap planning. By analyzing the sequence of clicks taken by users on a website, clickstream data can provide valuable customer insights and enhanced business intelligence. Other use cases include web movement investigation, statistical surveying, and programming testing (Hanamanthrao & Thejaswini, 2017). Additionally, AI and machine learning models are transforming how product roadmap planning is done. One key advantage of its implementation is the acceleration of timeline creation. Furthermore, AI and ML algorithms have already been implemented in other industries like traffic management and urban planning to make real-time decisions, based on data collected from GPS and sensors (Khare & Arora, 2024).

Capital One, the sponsor company, is a leading company in the banking industry with a great emphasis on information and technology. Specifically, the Agent Servicing team serves to enhance customer support by providing agents with the necessary tools to effectively assist customers. They provide services that seamlessly handle customer inquiries by enabling agents to access relevant real-time information such as customer account details, transaction histories, and service records. Currently, the platform does not have any integration with artificial intelligence.

Clickstream data has been used to generate AI insights for project and product planning, but it is less common in industries related to customer support platforms for large-scale digital services in the banking industry. For instance, historically, clickstream data has been utilized for optimizing websites and predicting user behavior. In a paper titled “Real-Time Clickstream Data Analytics and Visualization”, Hanamanthrao & Thejaswini (2017) highlighted the benefits of real-time data processing over traditional batch processing techniques for clickstream analysis. Data collected from an online learning platform website included course enrollment, navigation patterns, and time spent per page. The data was stored and processed using Apache Hadoop, an open-source big data storage platform. A real-time data pipeline was implemented using Apache Kafka for data streaming, Apache Spark for data processing, and ElasticSearch for data indexing and searching. Real-time processing led to the creation of immediate insights, such as the identification of popular courses and optimal navigation pathways on the portal. The results also demonstrated that real-time processing improved course recommendation accuracy and resource allocation, compared to traditional batch processing. Previous approaches required more time to process data before any insights could be applied. Manipulation of real-time data processing sets the groundwork for future predictive analytics related to improvements based on live user behavior. Furthermore, this approach allows for future implementation of ML models which create personalized recommendations.

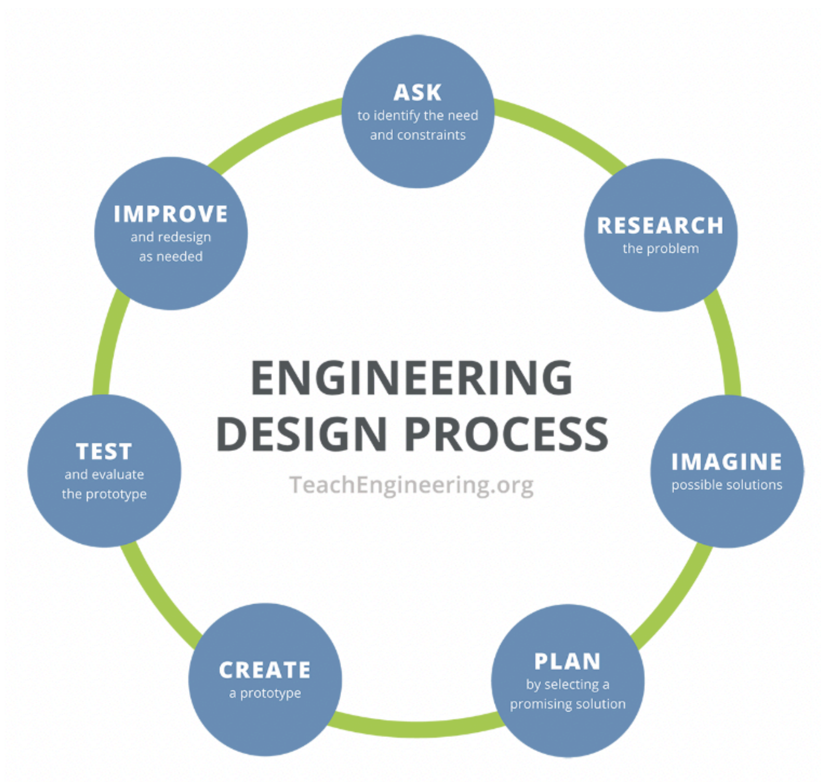
While Hanamanthrao & Thejaswini demonstrated the clear effectiveness of real-time processing, there was no implementation of AI or ML algorithms. Gumber et al.conducted a study, “Predicting Customer Behavior by Analyzing Clickstream Data”, which emphasized the importance of clickstream in understanding user behavior and applying it to develop a machine learning model. In this academic journal, they also reference Hanamanthrao & Thejaswini in the literature survey, noting that they were unable to collect data from multiple sources. Clickstream data was collected from an e-commerce site because shoppers often navigate through multiple pages before making a purchase decision. Unstructured data was obtained from Kaggle, an open-source platform providing datasets, containing information such as event time, event type (view, cart, purchase), and category information. Preprocessing was conducted before implementing the machine learning algorithm to make it usable. The machine learning model chosen, XGBoost, can handle large datasets well while processing them quickly with fewer computing resources. It also utilizes regularization techniques to prevent overfitting, a common machine learning behavior that can potentially give inaccurate predictions, more efficiently than other boosting algorithms. Using the Extreme Gradient Boosting (XGBoost) algorithm, customer behavior predictions were made with 85.9% accuracy and 91.04% recall (Gumber et al., 2021).

Though the industry in which the research was performed is vastly different from agent servicing platforms, similar principles could be applied to enhance customer support platforms.

“Neural Networks for Customer Classification Through Clickstream Analysis” written by Erika Severeyn, Alexandra La Cruz, Roberto Matute, and Juan Estrada in 2023 goes over how most organizations currently interpret large amounts of data within their databases and how they hope to move toward AI algorithms in the future. The article uses the term business intelligence (BI), which is the strategies, technologies, and tools that an organization uses to analyze data. They then use that information to try and gain insights to conduct future business decisions. By analyzing this data organizations can attempt to make predictions about customers behavior, market trends, and other business-impacting factors. The newest addition to companies BI’s has been the use of AI to break down these large data sets. Leveraging clickstream data is a valuable resource that can significantly aid businesses in enhancing their productivity through the implementation of artificial intelligence for data analysis. The main use of this so far has been to personalize online experiences for customers. The type of AI being used is an AI that has a neural network. They mimic having a human brain to learn as they get fed more information. By using these neural networks businesses can more accurately predict user behaviors. For the research done in the paper, they used Clickstream data from IMOLKO C.A.’s website. IMOLKO C.A. is a service company that helps businesses increase their profits, retain clients, and reduce the churn rate. The dataset was collected by Google Analytics which is a platform that can collect real-team user interactions. The researchers' goal was to see which users would become customers based on the dataset. They trained the neural network on the pre-processed dataset. They did five different training techniques, it was performed for 90% training and 10% testing, 80% training and 20% testing, 70% training and 30% testing, 60% training and 40% testing, and 50% training and 50% testing. The result was that the different techniques did not affect the AIs accuracy. It stayed consistent throughout all the training. The model demonstrated that it could accurately identify positive cases, cases where the user did become a customer, 85% of the time. It was much better at identifying when a user would not become a customer, achieving an accuracy of 91%.

Another article that we looked into was focused less on trying to make a profit and more on helping users correctly use their website. The article is called “A Machine Learning-based procedure for leveraging clickstream data to investigate early predictability of failure on interactive tasks” and it was written by Esther Ulitzsch, Vincent Ulitzsch, Qiwei He, and Oliver Lüdtke in 2022. They wanted to see if they could get an AI to predict whether or not a user would fail an interactive problem based on their actions on the website. The thought process was that there was sufficient information for predicting outcomes based on which actions they performed early on and how long it took them. They wanted to be able to predict the outcome as early and accurately as possible. This was done to see if there was a pattern in the way a user would attempt to solve a problem and their success rate. If there was, the website could be redesigned to try and raise the percentage of successful problem-solving. They tested users with two different problems in their study. They called these problems “Lamp Return” and “Meeting Rooms”. “Lamp return” involves going through an online shop to return a lamp. “Meeting Rooms” involved going through an online website to reserve rooms based on different requests.

The data from 6,791 “Lamp Return” problems and 6,629 “Meeting Rooms” problems were used for analysis. The success rate for both problems was around 50%. Those who failed generally did fewer actions and spent less time on the problems. The data was preprocessed to make it easier for the AI to understand when it was trying to predict outcomes. This researcher also used XGBoost to try and predict the outcome. The AI was trained on the data and it made its predictions based on many formulas set up by the researchers. The results were that the AI was able to achieve excellent classification performance for the “Lamp Returns” problem. The results were not as clear for the “Meeting Rooms” problem. This comes from the AI’s most predictive feature being time elapsed until action. This means how long the user would spend reading the problem before doing anything. It was much more of a telling feature for the “Lamp Returns” problem than it was for the “Meeting Rooms” problem. Overall though the AI was very accurate at predicting the outcome of the test very early on in the testing. The difference between the accuracy in the problems only shows how important the criteria the AI follows for its predictions are.



Section B. Engineering Design Requirements

This section outlines the core goals, objectives, design specifications, and constraints that guide the development of this project. It provides a structured framework for defining the problem space, informed by client needs, and outlines the conditions and limitations within which the design will operate. By clearly establishing these requirements, the project ensures alignment with stakeholder expectations and creates a roadmap for delivering a successful solution.

The engineering design requirements address several critical aspects that guide the development of the project. First, the Project Goals section describes the overarching goals from the client's perspective, focusing on the high-level outcomes the client expects. These goals do not provide specific details about the design but highlight the purpose and intent behind the project.

Next, the Design objectives focus on specific, measurable, and time-bound goals that define what the design will achieve. These objectives are drawn from the client's needs and translated into actionable tasks for the design, ensuring that the solution meets the expected functionality.

The Design Specifications and Constraints section lists the measurable and testable limitations that must be met for the design to be considered successful. It includes performance requirements, hardware capabilities, cost constraints, and data handling regulations, ensuring the design adheres to both functional and legal expectations.

B.1 Project Goals (i.e. Client Needs)

The project aims to help Capital One stakeholders make informed, data-driven decisions using clickstream data. This project will address the underutilization of clickstream data in roadmap planning and will empower stakeholders to gain actionable insights into user behavior. the primary goals of the project are:

- To leverage real-time clickstream data to provide AT-driven insights for platform optimization.
- To build a web application that mimics Capital One's Help Center and allows stakeholders to visualize data in real time.
- To automate the generation of recommendations for improving key metrics like Average Handle Time (AHT) and user engagement.
- To seamlessly integrate the insights and recommendations into JIRA, allowing for the automatic creation of epics or stories that align with AI-driven suggestions.

B.2 Design Objectives

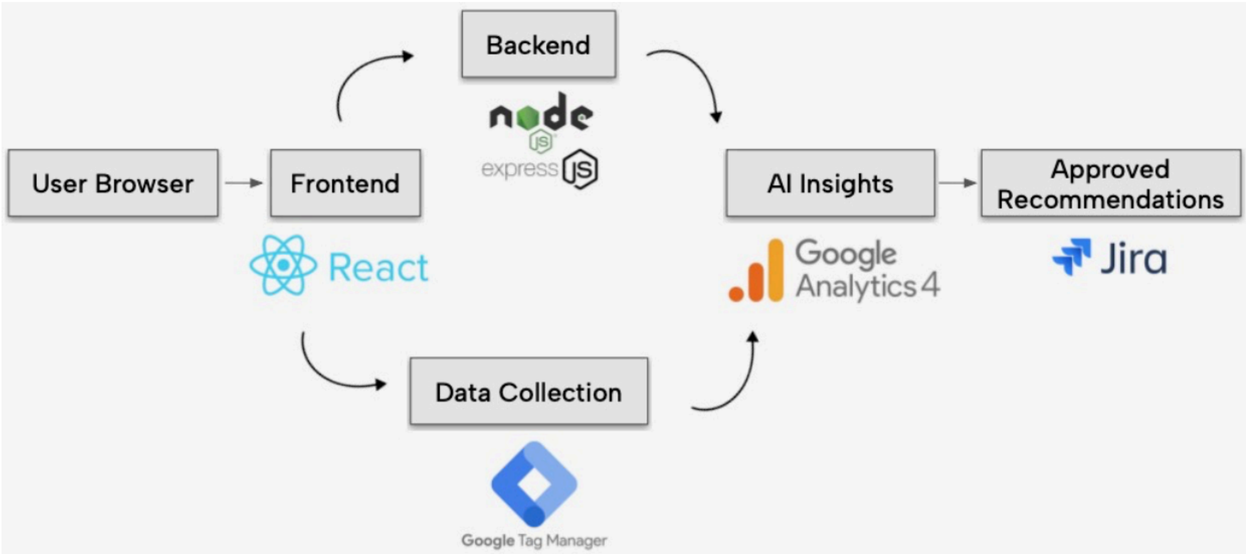
- The design will capture real-time clickstream data from users interacting with a web application that mimics the Capital One Help Center
- The design will provide AI-generated insights based on the clickstream data to answer queries such as, "How can I reduce AHT in the X container?" or "What are my critical features?"
- The design will integrate with JIRA via APIs to allow stakeholders to automatically create epics or stories based on the recommendations provided by the AI model.

- The design will offer a user-friendly, responsive interface that provides easy navigation, filtering, and data visualization using charts and graphs to help users analyze trends and performance metrics.
- The design will ensure that the platform operates effectively across different devices (desktop, mobile, tablet) to accommodate diverse users.

B.3 Design Specifications and Constraints

- The application must handle real-time clickstream data processing with a latency of no more than 5 seconds to ensure accurate insights
- The AI model must analyze clickstream data and generate recommendations with an accuracy rate of at least 85%. the system will utilize OpenAI's GPT for generating insights.
- The system must handle up to 1,000 concurrent users and process data efficiently without performance degradation. This ensures the platform's scalability for larger user bases.
- The application must interface with JIRA via REST APIs to create epics or stories automatically from AI recommendations within 3 seconds of acceptance.
- Clickstream data must be stored in a scalable database capable of handling data growth of at least 1GB/day without affecting system performance.
- The system must comply with Capital One's privacy regulations, ensuring that no personal identifiable information (PII) is exposed or stored. Data encryption will be implemented for both in-transit and at-rest data.
- The UI should allow for filtering of data by various parameters (e.g., date range, container type, user actions) and ensure responsiveness on mobile, tablet, and desktop devices. Performance tests will ensure UI responsiveness within 2 seconds across devices.
- The system should have an uptime of at least 99%, ensuring minimal downtime for continuous access to insights and recommendations.

Tech Stack Diagram:


Backend
User Browser
Frontend
AI Insights
Approved Recommendations
React
Google Analytics 4
Jira
Data Collection
Google Tag Manager

B.4 Codes and Standards

The design and implementation of our React-based customer service web application adheres to several important standards and codes that ensure technical consistency, user safety, accessibility, and interoperability across platforms. Below are the relevant codes and standards we have followed throughout development:

Standards

- W3C Standard – HTML5/CSS3 Web Content Standards
- The application complies with W3C HTML5 and CSS3 standards to ensure proper rendering across modern browsers, semantic markup for accessibility, and responsive design practices. This includes valid tag usage, CSS box models, and responsive layouts using Flexbox (App.css).

- ISO/IEC 40500:2012 – Web Content Accessibility Guidelines (WCAG) 2.0
- We considered WCAG 2.0 Level A/AA accessibility standards to improve usability for individuals with disabilities. This includes the use of semantic elements (<button>, <input>, etc.), descriptive alt text for icons, and adequate color contrast in the UI.

- IEEE 830-1998 – Software Requirements Specification Standard
- While not formally documented in IEEE 830 format, our functional components (App.js) and testing scripts (App.test.js) align with the intent of this standard by clearly defining expected inputs, outputs, and user interactions.

- ECMA-262 – ECMAScript Language Specification
- The JavaScript code, including the React component architecture, adheres to the ECMAScript 2021 (ES12) standard as enforced through the React and Node.js toolchains.

- NIST SP 800-53 Rev. 5 – Security and Privacy Controls (applicable to integration and analytics)
- While no user authentication was implemented, our backend design for future Google Analytics and JIRA integration considers NIST recommendations for data handling, least privilege access, and secure API token storage.

Codes

- General Data Protection Regulation (GDPR) – EU Regulation 2016/679
- Our app design anticipates future compliance with GDPR by avoiding unnecessary personal data collection and preparing for anonymized analytics integration via Google Analytics, in alignment with privacy-by-design principles.

- California Consumer Privacy Act (CCPA)
- Similarly, we consider CCPA implications for any future data collection involving California residents, ensuring that data is collected transparently and can be deleted upon user request.

- OSHA Code 1910.1200 – Hazard Communication (for internal development environment)
- Although not user-facing, we maintain safe development practices aligned with OSHA standards for screen-time ergonomics, workspace cleanliness, and data protection in shared environments.

Section C. Scope of Work

The scope of this project involves building an AI-powered web application to analyze clickstream data for platform performance recommendations, with integration into JIRA for project management. The key objectives include capturing and analyzing real-time data, generating actionable insights, and streamlining roadmap planning. This project is critical for platform stakeholders to make data-driven decisions to improve user engagement and other key metrics.

Boundaries for the project include a focus on web application development with specific deliverables and milestones as outlined below. The team will operate under the Agile methodology, utilizing iterative development to adapt as insights from earlier project phases are incorporated into subsequent stages. The team will work closely with the faculty advisor and project sponsor to ensure timely completion and adherence to the scope, preventing scope creep.

Stakeholder Involvement:

The project sponsor and faculty advisor play a crucial role in verifying and approving each stage of the project. Regular reviews and feedback sessions (weekly Zoom meetings) will be conducted to ensure that the project remains aligned with the expectations and requirements set by the stakeholders. This will help identify any potential issues early on, ensuring any necessary adjustments are made without deviating from the project's scope. Active stakeholder engagement will mitigate the risk of scope creep and help ensure that all deliverables meet the desired quality standards.

C.1 Deliverables

Project Deliverables:

- Web Application: A functioning web app with a user-friendly interface mimicking the Capital One Help Center page. The app will capture and display real-time clickstream data, providing actionable AI-driven insights.
- Clickstream Data Analysis: Real-time data processing and visualization using AI to recommend improvements, including metrics like Average Handle Time (AHT) and user engagement.
- JIRA Integration: Automatic creation of mock Epics and stories based on AI recommendations.
- AI/ML Recommendation Engine: Integration of OpenAI's API for generating insights from clickstream data.
- Final Prototype: A complete, functional prototype that can be demonstrated for review.

Academic Deliverables:

- Team Contract
- Project Proposal
- Preliminary Design Report
- Fall Poster and Presentation (November 2024)
- Final Design Report
- Capstone EXPO Poster and Presentation (Spring 2025)

Risks and Mitigations:

- Access to campus: Some team members may not have consistent access to campus facilities due to living off-campus, but the majority of work can be completed remotely.
- Remote Work: Resources like shared drives, GitHub, and remote collaboration tools (VS Code, JIRA, and Slack) will be used to mitigate risks.
- Third-party vendor delays: The use of existing APIs and software mitigates the need for extended lead times or delays related to third-party ordering.

C.2 Milestones

Milestone	Estimated Completion Date

Team Contract - Establish a foundation for team collaboration and communication. The document lists individual team member strengths, roles, and responsibilities, time commitments, and the communication/meeting structure.	September 6, 2024
Project Proposal - Frame the project early on by outlining key details such as background information, objectives, and the unmet engineering need being addressed. Includes sections on background, literature review, project goals, objectives, constraints, project scope, deliverables, organizational structure, and a proposed timeline.	October 11, 2024
Prototype Web Application Build - Develop a basic web application version to mimic the Capital One Help Center page, integrating key features and functionalities to create a working model for testing and feedback.	November 15, 2024
Fall Design Poster - Present preliminary design results in a public forum at the end of the semester, where team members can showcase their work to sponsors, faculty, administrators, and peers.	November 15, 2024
Preliminary Design Report - Provide a comprehensive summary of the team's design efforts over the semester, with more detail than the project proposal.	December 9, 2024
Data Collection and AI Integration - Implement a system for collecting relevant data, followed by integrating AI/ML models to analyze and interpret the data, providing insights for future development.	February 2025

AI Recommendations in JIRA - Deploy the AI-powered solution, enabling it to analyze clickstream data and provide actionable recommendations. These will be integrated with JIRA to create mock Epics and stories for roadmap planning.	March 2025
Final Design Report Submission - Deliver a detailed report summarizing the team's work, including final design, implementation results, and project outcomes, showcasing the full scope of the project.	May 2025
Capstone Design Expo - Present the final project at a public expo, demonstrating the completed work to industry sponsors, faculty, and peers, emphasizing technical achievements and project outcomes.	April 24-25, 2025

C.3 Resources

1. Software:
    a. Visual Studio Code (VS Code) – Used as the primary Integrated Development Environment (IDE) for front-end and backend development.
    b. Node.js & npm – Enabled project scaffolding, dependency management, and build scripts necessary for React development.
    c. React.js – Framework used to build the modular, component-based user interface.
    d. Jest + React Testing Library – Used for unit and UI testing, as set up in App.test.js and setupTests.js.
2. Version control:
    a. Git & GitHub – Source code was managed using Git with remote hosting and collaboration enabled via GitHub. Branching and pull request workflows were used during development.
3. Cloud Computing Services:
    a. Vercel – Used for hosting the live version of the React application and deploying serverless backend functions.
    b. Google Cloud Platform (GCP) – use in scalable clickstream data processing and storage, depending on load and analytics demand.
4. APIs and Libraries:

   a. OpenAI API – Integrated to generate real-time, AI-driven recommendations for UX improvements and task automation based on analytics data.
   b. JIRA REST API – Used to automate issue creation (e.g., Stories, Epics) based on AI suggestions. Integration supported features like filtering, searching, and creating issues programmatically.
   c. Axios – Used for client-server communication and proxy routing to backend APIs.
5. Analytics & Data Resources:
   a. Google Analytics 4 (GA4) – Implemented to track user behavior, gather clickstream data, and drive data-informed decision-making via AI insights.
   b. Clickstream Datasets (Synthetic or Sample) – Used to test AI models and simulate real-time behavior in the absence of live user data.

At this point, all necessary tools have been secured. No further purchases are anticipated. However, depending on future scale or production demands (e.g., increased traffic or additional feature sets), we may consider allocating funds toward cloud storage credits, API usage expansions, or commercial licenses for additional services.

Section D. Concept Generation

Several design concepts were explored to address the challenges of utilizing clickstream data for roadmap planning, performance optimization, and intelligent task automation. Each concept was evaluated based on its alignment with the project's objectives, feasibility of implementation, user interactivity, and integration potential with external tools such as JIRA and OpenAI.

The first design concept involved building a fully modular web application using React, integrated with AI services and a backend capable of processing real-time clickstream data. This approach aimed to provide stakeholders with actionable insights directly within the platform, including visual analytics dashboards, AI-driven recommendations to improve metrics like Average Handle Time (AHT), and automated task generation via JIRA integration. This concept offered strong customization potential and scalability, allowing future expansion of features or services. However, it required significant development effort and technical resources, particularly in creating a user-friendly interface and ensuring secure and effective integration with external APIs. The risk of integration failures or inaccurate AI suggestions was a key consideration, along with the time investment needed to build and test such a complex system.

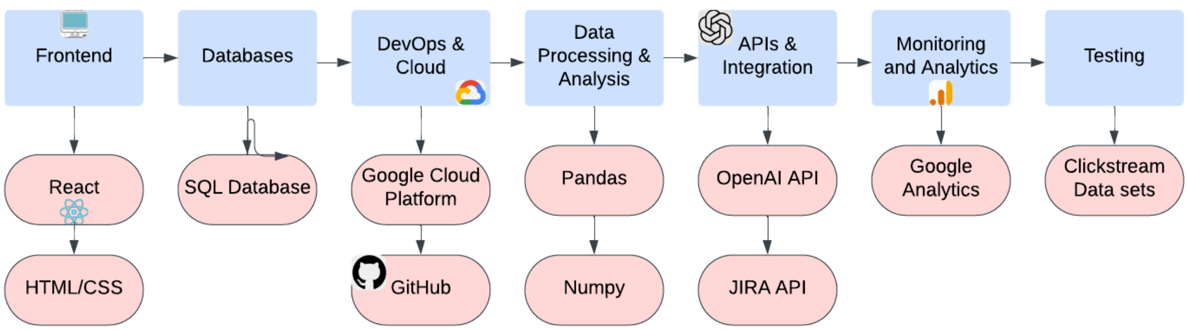
The second concept centered on a backend-focused system that processes clickstream data and delivers insights through external visualization platforms such as Tableau or Microsoft Power BI. This design prioritized backend data analysis and reporting efficiency while forgoing a custom frontend interface. Insights generated by AI models would be shared with stakeholders through familiar tools, reducing the need for extensive frontend development. While this solution offered quicker development and leveraged robust third-party visualization capabilities, it came with limitations in interactivity and customization. Users would be restricted to predefined dashboards, and real-time exploration of data would be limited. There were also risks associated with maintaining compatibility across different platforms and the inability to tailor the insight presentation to specific stakeholder needs.

The third design concept proposed an AI-powered interactive dashboard with a lightweight custom frontend. This approach bridged the gap between Concepts 1 and 2 by offering a simplified yet user-friendly interface for exploring AI-generated insights. Stakeholders could filter, sort, and view trends in clickstream data in real time, making it easier to interpret and act on analytics without needing the depth of a full-scale application. The key advantage of this concept was its balance of usability and simplicity, allowing faster deployment than a fully modular system while still maintaining a degree of user interactivity. However, it did not offer the same level of scalability or backend processing power as the more advanced solutions, and might fall short of meeting all user needs for complex data exploration or integration.

The fourth and most user-centric concept was the development of a natural language query system, enabling stakeholders to ask questions such as “How can I improve AHT?” and receive instant AI-generated responses. This concept prioritized accessibility and simplicity, providing users with direct answers rather than requiring them to interpret dashboards or sift through large reports. It was designed to serve non-technical users who may not be familiar with data analysis tools. While this solution offered a highly intuitive experience, it lacked visual support for complex data trends and required sophisticated natural language understanding capabilities to accurately interpret and respond to varied queries. Risks included ambiguity in query interpretation, potentially inaccurate AI responses, and a lack of visual validation for the insights presented.

Ultimately, after analyzing the pros, cons, and risks of each concept, the team decided to implement a hybrid of the first and fourth designs. This combined the flexibility and robustness of a modular React application with the intuitive ease-of-use of a natural language AI query interface. This hybrid approach provided stakeholders with both dynamic visual tools and the ability to engage directly with AI for insights, offering a comprehensive solution that balances technical depth with user accessibility.

Below is the original tech stack diagram when the project was still in development. We have since updated it because we are using different technologies.



Section E. Concept Evaluation and Selection

To systematically evaluate the four proposed design concepts, we used a decision matrix based on a defined set of selection criteria aligned with the project's core objectives and client expectations. The criteria included real-time data processing, scalability, ease of implementation, AI functionality, and user interactivity. These factors reflect both technical performance goals and usability considerations essential for a data-driven, customer-facing web application.

Each criterion was assigned equal weight for simplicity, as our client emphasized the importance of a balanced solution that delivers real-time insights while remaining user-friendly and adaptable for future growth. Metrics were scored on a scale of 1 to 10 based on relative performance in each area, using a combination of research, prototype observations, and technical feasibility assessments.

Real-time data processing reflects the system's ability to capture and analyze user interactions instantly. Scalability considers whether the architecture can support growing data volume and new features. Ease of implementation estimates the development effort and integration complexity. AI functionality measures the system's capability to generate meaningful, actionable recommendations. UI and interaction evaluates the user experience, including accessibility, intuitiveness, and visual design.

The first concept—a fully modular web application with integrated AI and JIRA automation—scored the highest overall. It offered strong capabilities across all metrics, particularly in interactivity, flexibility, and alignment with project goals. While it involves a longer development timeline, its architectural robustness and adaptability make it a sustainable and comprehensive solution.

The second concept, a backend-focused system using external dashboards, excelled in implementation speed and backend efficiency. However, its lack of a user-facing interface makes it less suitable for stakeholders who need direct access to insights. It was considered more appropriate for internal analytics use cases, not customer-facing scenarios.

The third concept, a lightweight AI-powered dashboard, performed well in usability and insight delivery. It offered a practical middle ground but fell short in scalability and backend processing power compared to other options. It would be ideal for MVPs or limited-scope deployments.

The fourth concept, the AI-assisted query system, presented an innovative and user-friendly interface through natural language interaction. However, it lacked robust visualization and required advanced NLP capabilities, making it a higher-risk solution for this stage of development.

Based on the evaluation results, we selected the modular web application with AI integration as the final design concept. Its flexibility, real-time insight delivery, and seamless integration with tools like JIRA support the long-term goals of the project and provide a scalable platform for performance monitoring and roadmap planning.

Table 1. Decision Matrix.

	Concept 1: Modular Web App	Concept 2: Backend + Reports	Concept 3: AI Dashboard	Concept 4: Query System
Real time processing	9	7	7	5
Scalability	8	9	6	6
Ease of implementation	6	8	7	7
AI functionality	8	7	7	6
UI and Interaction	9	4	8	6
Total Score	40	35	35	30

Section F. Design Methodology

The design methodology for this project follows an iterative engineering design process, ensuring that the final system fulfills all project objectives and specifications while addressing the client's functional needs. This iterative approach includes continuous evaluation, refinement, and validation through a combination of computational modeling, experimental testing, and stakeholder-driven feedback loops. The methodology is grounded in core engineering principles of modularity, scalability, and usability, which guide every phase of development—from high-level architecture to detailed integration and testing.

The system is designed to deliver actionable, real-time insights derived from clickstream data, generate AI-based recommendations, and automate the creation of development tasks through seamless JIRA integration. Verification confirms the system meets technical specifications, while validation ensures it performs effectively in a real-world context and meets client expectations.

F.1 Computational Methods

The core application was developed using React.js, with a component-based structure allowing reusable, scalable UI elements. Styling was handled with custom CSS files, and image assets were integrated into the layout for visual clarity and responsiveness.

For analytics, we used Google Analytics 4 (GA4) to track clickstream data, capturing user behavior across the site. This data was fetched through a serverless API endpoint hosted on Vercel, where we created a custom backend function to securely retrieve and relay data.

AI-based insights were generated using the OpenAI API, which received pre-processed analytics data and returned improvement suggestions. These suggestions were rendered on the frontend and categorized into labels such as [Bug], [Task], [Story], or [Epic].

To create JIRA issues from these AI suggestions, we implemented a secure proxy-based integration using serverless functions on Vercel. Requests from the frontend were routed through backend endpoints (/api/create-issue, /api/search-issues) that authenticated with the JIRA REST API and executed actions such as creating or filtering issues.

Testing procedures included manual testing of user flows and integration checks using browser dev tools and console logging. We also utilized React Testing Library with Jest (via App.test.js) for unit testing of UI components.

F.2 Experimental Methods

Experimental methods focused on validating user experience and real-world functionality in a browser environment. The deployed site was tested under real usage conditions, simulating user search behavior, button interaction, and JIRA issue submission.

We conducted informal usability sessions to observe how users interacted with help topics, search features, and AI-generated suggestions. Metrics such as user click paths, engagement times, and search bar usage frequency were tracked through Google Analytics.

Throughout testing, we collected observations and feedback to refine the layout and improve clarity. For example, we adjusted banner spacing, button responsiveness, and the search interface based on real-time observations of how users navigated the system.

F.3 Architecture/High-level Design

The high-level architecture consists of a React frontend, integrated with Google Analytics for behavioral tracking, the OpenAI API for analysis and recommendation generation, and the JIRA REST API for task automation.

The app is deployed on Vercel, which hosts both the frontend and the serverless backend endpoints. These backend functions serve as secure intermediaries between the frontend and external services (OpenAI and JIRA), handling authentication and request formatting.

This architecture supports modular updates, minimizes security risks through API proxying, and ensures scalability via Vercel's built-in serverless infrastructure. No dedicated backend frameworks or databases were used, as the application depends on API-driven services and cloud-hosted analytics.

F.5 Validation Procedure

Validation of the final design focused on confirming that the system met both the technical and functional needs of the client. In early April, a live demonstration was conducted to showcase core features including: AI-driven analytics suggestions, search interaction, dynamic UI updates, and automated JIRA issue creation.

Client feedback was gathered through live observation and follow-up discussions, with particular attention to clarity of the interface, speed of AI response, and usability of the JIRA automation process. We recorded notes on stakeholder reactions and specific improvement suggestions.

Quantitative validation was supported by checking:

- Google Analytics data accuracy
- OpenAI response clarity
- Success status of JIRA issue creation requests (via console logs and API response codes)

The validation phase confirmed that the system met its design objectives, offering actionable insights, streamlined workflows, and an intuitive user experience—all within a serverless, API-integrated architecture.

Section G. Results and Design Details

The final design of the customer service web application successfully integrates clickstream analytics, AI-driven insight generation, and automated issue creation using a modular, API-based architecture. Through iterative development and testing, we achieved a system that delivers real-time user behavior tracking, generates actionable suggestions using OpenAI, and seamlessly creates JIRA tasks based on those insights. This section highlights the computational, experimental, and design results that demonstrate how the system meets all outlined design specifications and objectives.

The identified design challenge—building a smart and scalable help center interface capable of analyzing clickstream behavior and automating roadmap planning—was addressed using a combination of client-side interactivity, serverless backend endpoints, and cloud-based analytics and AI services. The final deliverables include a fully deployed web application with interactive UI components, dynamic search and navigation features, real-time data collection through Google Analytics, and JIRA integration through backend functions hosted on Vercel.

G.1 Modeling Results

Although no physical modeling or simulation was used, logical and system architecture modeling was essential. A dataflow model was developed to track how clickstream data traveled from the frontend (user interactions) to backend analytics (Google Analytics), then to OpenAI for recommendations, and finally to the JIRA REST API for issue creation.

A high-level diagram illustrates the flow:

- React Frontend → captures interactions (button clicks, search input)
- Google Analytics 4 → collects clickstream data in real time
- Vercel Serverless Function → fetches analytics data and relays it to OpenAI
- OpenAI API → analyzes data and returns improvement suggestions
- Another Serverless Function → takes suggestions and creates JIRA issues via REST API

This modular dataflow enables scalability and security by isolating API keys in backend environments while maintaining a smooth user experience on the frontend.

G.2 Experimental Results

Experimental validation involved manual interaction testing and simulated usage flows. Testing was conducted using browser dev tools, console output, and API response logs. We validated that:

- Google Analytics successfully captured all key interactions, including page loads, banner button clicks, and search inputs.
- OpenAI returned relevant suggestions when passed clickstream summaries, labeling them appropriately with tags such as [Bug], [Task], [Story], and [Epic].
- JIRA API accepted authenticated POST requests from the serverless function and correctly created issues with the expected title, description, and issue type.

Sample response logs confirmed 100% success for authenticated issue creation when the required fields were correctly passed.

G.3. Prototyping and Testing Results

The prototype was built and deployed using React.js, and tested through several iterations. Key refinements based on user feedback included:

- Adjusting layout spacing in the top and bottom banners for cleaner alignment
- Improving search bar usability by adding placeholder text and making button interactions more responsive
- Verifying that button clicks corresponded to accurate Google Analytics events
- Displaying AI-generated suggestions in a structured layout for clarity
- Ensuring JIRA issue creation was triggered only upon user approval of a recommendation

React Testing Library was used to write unit tests verifying UI rendering, such as confirming the presence of certain labels and buttons. Final tests demonstrated the system's stability across Chrome and Safari on both desktop and mobile formats.

G.4. Final Design Details/Specifications

Design Specification	Target	Achieved
Clickstream tracking	Must track user interaction events	Google Analytics 4 events confirmed via dashboard
AI insight generation	Suggestions based on clickstream	Returned categorized suggestions using OpenAI
Issue automation	Create issues in JIRA via API	Verified creation via REST API (POST 201 response)
UI responsiveness	Accessible on desktop and mobile	Responsive layout in testing
Deployment	Publicly hosted	Live deployment on Vercel
Usability	Minimal steps to insights/tasks	Search, button click, and approve recommendation

Section H. Societal Impacts of Design

H.1 Public Health, Safety, and Welfare

Our project improves digital customer service by enabling faster, clearer, and more efficient communication through AI-generated recommendations and real-time data tracking. By reducing user friction and optimizing interfaces for ease of use, the system promotes a stress-free experience that supports user mental well-being and digital literacy.

To prioritize user safety and data protection, our design adheres to widely accepted privacy and data handling standards:

- Data Privacy Compliance: Our application does not collect or store Personally Identifiable Information (PII), ensuring alignment with regulations like the General Data Protection Regulation (GDPR) and the California Consumer Privacy Act (CCPA).
- Secure API Communication: All data transfers to OpenAI and JIRA occur through authenticated and secure serverless proxy functions hosted on Vercel. API tokens are not exposed to the frontend, maintaining a secure interaction model.
- Google Analytics Safeguards: Only anonymized interaction data (e.g., button clicks, search usage) is collected, with strict controls over what information is tracked, avoiding any user-specific identifiers.

H.2 Societal Impacts

This project makes AI-enhanced analytics accessible to non-technical users by delivering a simple, intuitive interface for customer service platforms. By allowing stakeholders to interact with recommendations and search functionality in a natural and user-friendly way, we lower the barrier for understanding complex data.

The integration of automation—particularly in suggesting and generating JIRA tasks—reduces repetitive work for support teams and product managers. While this shift toward automation can change traditional workflows, it highlights the importance of equipping teams with digital tools and training, supporting a workforce transition into more analytical and strategic roles.

Ultimately, this platform encourages a broader cultural shift toward data-informed decision-making and accessible AI for non-engineering teams.

H.3 Political/Regulatory Impacts

While the system does not process sensitive financial data or user accounts, it still interacts with behavioral data through clickstream tracking. Therefore, it was designed with privacy-by-default principles to anticipate evolving regulatory landscapes.

By adhering to the GDPR and CCPA frameworks—avoiding PII collection, using anonymized metrics, and storing sensitive credentials securely in environment variables—we preemptively align with global and national data protection trends. This proactive design makes the system well-positioned to comply with any future regulatory updates regarding digital analytics or AI integration in customer-facing tools.

H.4. Economic Impacts

The system has the potential to increase operational efficiency for customer service and product teams by surfacing high-priority issues based on user behavior. Automating JIRA issue creation reduces time spent on manual reporting and triage, enabling quicker responses and resource allocation.

From an organizational perspective, implementing such a tool could reduce Average Handle Time (AHT), increase customer satisfaction, and streamline internal processes—all of which translate into cost savings and higher productivity. Moreover, by leveraging free-tier or cost-efficient APIs (OpenAI, Google Analytics, and Vercel functions), development and operational costs are minimized, making the solution economically accessible to small and mid-sized teams.

H.5 Environmental Impacts

The decision to build a serverless application deployed on Vercel helps reduce environmental impact by avoiding always-on infrastructure. Serverless functions are triggered only when needed, reducing energy consumption compared to traditional servers.

Additionally, by relying on cloud-native services (such as Google Analytics and OpenAI's hosted models), we minimize the need for physical hardware, reduce on-premise resource demands, and align with broader trends in environmentally conscious computing. While AI inference does consume energy, our usage is minimal and efficient—focused on short, targeted requests rather than continuous model training.

H.6 Global Impacts

Although developed as a customer service prototype, this system offers a replicable model for global organizations looking to leverage user interaction data for actionable insights. Its low-barrier implementation (React + Vercel + APIs) makes it scalable and adaptable across industries and regions.

By emphasizing ethical use of AI, privacy compliance, and transparency in data handling, the system contributes to setting a responsible standard for global digital services. It supports the advancement of data-driven decision-making worldwide while encouraging organizations to adopt fair and transparent design principles.

H.7. Ethical Considerations

Ethical responsibility was central to our design. We ensured that:

- No personal data is collected or inferred, respecting user privacy.
- AI-generated suggestions are visible and editable by users before any JIRA issues are created, preventing blind reliance on automation.
- All integrations are secured, and sensitive data (API keys, user input) is never exposed on the frontend.

We are mindful of potential misuse of AI (e.g., automating the creation of misleading or low-priority issues) and designed the system to require human oversight before any critical task is generated.

This approach ensures that the AI serves as a supportive tool rather than a decision-maker, preserving user agency and accountability.

Section I. Cost Analysis

This project was developed entirely as a web-based software application using open-source tools and publicly available APIs. As such, no physical hardware or experimental setup was required, and there were no manufacturing or material expenses incurred. The project focused on the development, deployment, and integration of a React-based frontend with serverless backend functions and external services including Google Analytics, OpenAI, and JIRA.

Resource/Tool	Description	Cost	Notes
Visual Studio Code	Code editor used for frontend and backend development	$0	Open-source IDE
React.js	JavaScript library for building the frontend	$0	Open-source MIT license
Vercel (Free Tier)	Hosting for frontend and backend serverless functions	$0	Free tier sufficient for testing and demo deployment
Google Analytics 4	Used for tracking clickstream and behavioral data	$0	Free tier supports GA4 integration and event tracking
OpenAI API	Used for generating AI-based recommendations	$20	Limited API calls used; assumed within low monthly use
JIRA (Atlassian Cloud)	Used for task creation via REST API integration	$0	Free plan available for up to 10 users
Postman	Used for API testing	$0	Free plan used during development
GitHub (Free Plan)	Version control, code collaboration, and repository hosting	$0	Used to track code and manage branches

Total Cost: $20

All tools used were either open-source, free for educational use, or used within the limits of their free tiers. The only cost came from OpenAI API usage, which remained minimal due to testing with a small number of requests.

I.2 Development Labor Estimate (for Commercial Context)

If the project were to be scaled into a production-ready commercial product, development labor would represent the primary cost component. Below is a high-level estimate of labor costs based on a small software development team.

Role	Hours Estimated	Hourly Rate	Total Cost
Frontend Developer	80	$40/hr	$3,200
Backend/API Developer	60	$45/hr	$2,700
QA Tester	20	$30/hr	$600
UI/UX Designer (Figma)	15	$35/hr	$525
Total Estimated Labor	-	-	$7,025

This estimate assumes an 8–10 week project cycle for an MVP (minimum viable product) version of the app.

I.3 Bill of Materials (N/A)

Since the project involved no physical components or hardware prototyping, there was no Bill of Materials (BOM) required. All development was completed in a digital environment using virtual tools and cloud-based services.

In summary, the project incurred no direct material or software costs, with all work completed using free resources and services. If developed for commercialization, the largest cost driver would be labor, estimated at approximately $7,000 for MVP-level deployment, not including marketing, customer support, or infrastructure scaling.

Section J. Conclusions and Recommendations

The design process for this project began with the central question: How can we leverage AI to enhance the efficiency of a webpage? Following this, we identified the necessary resources and established a timeline to guide the project. The first critical step was obtaining data for the AI model to learn from, and we determined that Clickstream data would be an ideal source. Clickstream represents the sequence of links clicked by a user on a website, providing valuable insights into user behavior. Since Clickstream requires a website to gather this data, and since we do not have access to an existing site, we have developed our own.

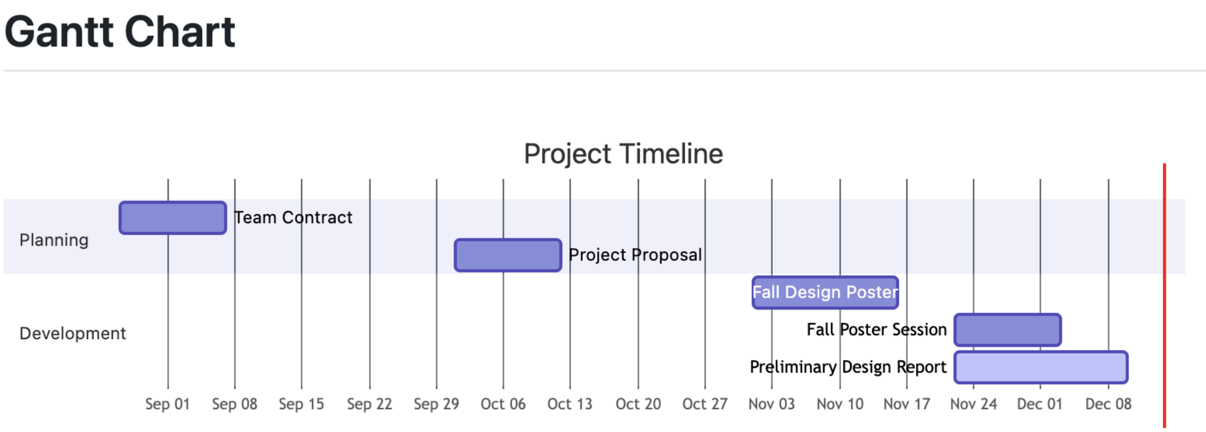
We trained the AI using the Clickstream data, with the goal of the AI autonomously creating new epics and stories within JIRA, a team management tool, which can then be reviewed by teams for potential implementation. Epics and stories are the terms used in JIRA to categorize and track tasks.

This plan represents the core concept behind our project design. However, we encountered challenges when deciding on the optimal resources and technologies for implementation. To address these issues, we met with mentors from both VCU and Capital One, whose advice has been extremely useful in refining our approach. With their advice, we were able to meet the project's primary goals and objectives effectively.

Looking ahead, there is significant potential for further development, particularly in the application of AI. While websites and JIRA integrations may have limitations in prototype form, AI has the capacity for continuous improvement. An interesting question to explore in future iterations is how the AI's suggestions might differ if it were trained using alternative methods. This would require extensive testing to determine the most effective approach. If the AI model we develop does not meet our expectations, the project can be continued by another group. This team would take over our website, either collecting additional data for further AI training or employing a different approach to refine the AI's performance. Our resources, including code and documentation, will be made available through GitHub for easy handover.

Another step in AI development could be having the AI create prototype versions of the website based on its insights. This would cut down on the workload and decision making for teams. Decision making would be easier since the team could see a prototype of what the new version of the website would look like. The workload would be less since the team would have the start of the code needed and would only need to modify it to fit their needs.

Appendix 1: Project Timeline



Appendix 2: Team Contract (i.e. Team Organization)

Step 1: Get to Know One Another. Gather Basic Information.

Task: This initial time together is important to form a strong team dynamic and get to know each other more as people outside of class time. Consider ways to develop positive working relationships with others, while remaining open and personal. Learn each other's strengths and discuss good/bad team experiences. This is also a good opportunity to start to better understand each other's communication and working styles.

Team Member Name	***Strengths each member bring to the group***	***Other Info***	***Contact Info***
Bindi Patel	Outgoing, willing to hear others' ideas, organized, JAVA, Python	interested in cybersecurity, has worked on web design (HTML, CSS, JavaScript)	(540)905-3849 patelb10@vcu.edu
Priya Choudhary	communication, organized, timely, C, Python, Java, PyCharm	Experience with front-end applications, mostly proficient in C and Linux environments, but has experience with Java and python	(703)731-5361 choudharyp2@vcu.edu
Carissa Trieu	Organization, Java, Python, Communication	Interest in data science, experience with Python from VIP, also comfortable with Java	(804)664-5277 trieuc3@vcu.edu
Ivan Emdee	Java, Flexible, Making diagrams,	Hoping to become a software engineer, in the process of learning other languages	(804)245-2593 emdeein@vcu.edu

Other Stakeholders	***Notes***	***Contact Info***
Thomas Gyeera	To get started, I would like for you guys to do some research on articles on related topics and previous research that has been done on this topic. summarize them and understand them so that they can help with your project proposal. you can expect research to also be sent from me to assist. you can expect to see the syllabus by	gyeeratw@vcu.edu

	the end of the weekend. So far I am happy with how prepared you guys were for the meeting today.	
Mahesh Nair Tyler Jordan Emily Croxall	Initial meetings with the team have shown us that you all have a lot of energy and are very engaged with the project. The team came prepared with questions and left the meeting with a clear path forward and action items.	mahesh.bahulleyannair@capitalone.com tyler.jordan@capitalone.com emily.croxall@capitalone.com

Step 2: Team Culture. Clarify the Group's Purpose and Culture Goals.

Task: Discuss how each team member wants to be treated to encourage them to make valuable contributions to the group and how each team member would like to feel recognized for their efforts. Discuss how the team will foster an environment where each team member feels they are accountable for their actions and the way they contribute to the project. These are your Culture Goals (left column). How do the students demonstrate these culture goals? These are your Actions (middle column). Finally, how do students deviate from the team's culture goals? What are ways that other team members can notice when that culture goal is no longer being honored in team dynamics? These are your Warning Signs (right column).

Resources: More information and an example Team Culture can be found in the Biodesign Student Guide "Intentional Teamwork" page (webpage | PDF)

Culture Goals	***Actions***	***Warning Signs***
Being on time	Text in the group chat to remind/decide on meeting times	- text if a date/time doesn't work for you or if you will be missing a meeting. - your responsibility to get updated on what was missed in the meeting
Thursday's at 6 PM will be dedicated to a group status update	- Follow agile principles: update what was done during the week, and what you expect to get done the next week, and identify any roadblocks that you need assistance with. - Set reasonable deadlines and note when an extension is needed	- Student shows up for weekly meeting with no considerable work done - The student doesn't update the group
records for each group member	- a Google doc that will be updated each meeting with information on what each team member plans/has accomplished - every meeting the scribe is rotated	- The scribe forgets to update the doc

Step 3: Time Commitments, Meeting Structure, and Communication

Task: Discuss the anticipated time commitments for the group project. Consider the following questions (don't answer these questions in the box below):

- What are reasonable time commitments for everyone to invest in this project?
- What other activities and commitments do group members have in their lives?
- How will we communicate with each other?
- When will we meet as a team? Where will we meet? How Often?
- Who will run the meetings? Will there be an assigned team leader or scribe? Does that position rotate or will same person take on that role for the duration of the project?

Required: How often you will meet with your faculty advisor advisor, where you will meet, and how the meetings will be conducted. Who arranges these meetings?
See examples below.

Meeting Participants	***Frequency Dates and Times / Locations***	***Meeting Goals Responsible Party***
Students Only	As Needed, On Facetime or normal phone call, Zoom if screen sharing	Update group on day-to-day challenges and accomplishments
Students Only	Every Thursday, in the library during normal lab time (if possible)	Actively work on the project
Students + Faculty advisor	Every Friday at noon (Room E4242)	Update the faculty advisor and discuss any questions
Project Sponsor - Tyler Jordan Emily Croxall	Every Friday at noon - can cancel if no topics are needed We will create a recurring Zoom meeting and send an invite to the team.	Help the team with project requirements and possibly technical questions throughout the duration of the project.

Step 4: Determine Individual Roles and Responsibilities

Task: As part of the Capstone Team experience, each member will take on a leadership role, ***in addition to*** contributing to the overall weekly action items for the project. Some common leadership roles for Capstone projects are listed below. Other roles may be assigned with approval of your faculty advisor as deemed fit for the project. For the entirety of the project, you should communicate progress to your advisor specifically with regard to your role.

- **Before meeting with your team**, take some time to ask yourself: what is my "natural" role in this group (strengths)? How can I use this experience to help me grow and develop more?
- **As a group,** discuss the various tasks needed for the project and role preferences. Then assign roles in the table on the next page. Try to create a team dynamic that is fair and equitable, while promoting the strengths of each member.

Communication Leaders

Suggested: Assign a team member to be the primary contact for the client/sponsor. This person will schedule meetings, send updates, and ensure deliverables are met.

Suggested: Assign a team member to be the primary contact for faculty advisor. This person will schedule meetings, send updates, and ensure deliverables are met.

Common Leadership Roles for Capstone

1. **Project Manager:** Manages all tasks; develops overall schedule for project; writes agendas and runs meetings; reviews and monitors individual action items; creates an environment where team members are respected, take risks and feel safe expressing their ideas.
   Required: On Edusourced, under the Team tab, make sure that this student is assigned the Project Manager role. This is required so that Capstone program staff can easily identify a single contact person, especially for items like Purchasing and Receiving project supplies.
2. **Logistics Manager:** coordinates all internal and external interactions; lead in establishing contact within and outside of organization, following up on communication of commitments, obtaining information for the team; documents meeting minutes; manages facility and resource usage.
3. **Financial Manager:** researches/benchmarks technical purchases and acquisitions; conducts pricing analysis and budget justifications on proposed purchases; carries out team purchase requests; monitors team budget.
4. **Systems Engineer:** analyzes Client initial design specification and leads establishment of product specifications; monitors, coordinates and manages integration of sub-systems in the prototype; develops and recommends system architecture and manages product interfaces.
5. **Test Engineer:** oversees experimental design, test plan, procedures and data analysis; acquires data acquisition equipment and any necessary software; establishes test protocols and schedules; oversees statistical analysis of results; leads presentation of experimental finding and resulting recommendations.
6. **Manufacturing Engineer:** coordinates all fabrication required to meet final prototype requirements; oversees that all engineering drawings meet the requirements of machine shop or vendor; reviews designs to ensure design for manufacturing; determines realistic timing for fabrication and quality; develops schedule for all manufacturing.

Team Member	*Role(s)*	*Responsibilities*
Priya Choudhary	Project Manager	● Keep a detailed record of meeting notes and share them with the group ● creates an environment where team members are respected ● develops the overall schedule for the project ● writes agendas and runs meetings
Carissa Trieu	Test Engineer	● Oversee project design, test plan, procedures, and data analysis ● Establish test protocols and schedules ● Lead presentation of recommendations and results
Bindi Patel	Logistics & Financial Manager	● Primary contact with advisors (updating them, etc) ● Record meeting minutes ● Handles the research/comparison of purchases ● Responsible for keeping track of the team budget
Ivan Emdee	Systems Engineer	● Analyze the client's initial designs to come up with a prototype ● Oversee the development of subsystems ● Recommend system architecture and manage product interface

Step 5: Agree to the above team contract

Team Member: Priya Choudhary *Signature: Priya Choudhary*

Team Member: Carissa Trieu *Signature: Carissa Trieu*

Team Member: Bindi Patel *Signature: Bindi Patel*

Team Member: Ivan Emdee *Signature: Ivan Emdee*

References

[1] VCU Writing Center. (2021, September 8). *APA Citation: A guide to formatting in APA style.* Retrieved September 2, 2024. https://writing.vcu.edu/student-resources/apa-citations/

[2] Teach Engineering. *Engineering Design Process*. TeachEngineering.org. Retrieved September 2, 2024. https://www.teachengineering.org/populartopics/designprocess

[3]J. Electrical Systems 20-2 (2024):1600-1608.Real-Time Clickstream Analytics with Apache. Retrieved October 10, 2024. https://www.researchgate.net/profile/Tulshihar-Patil/publication/382193441_Real-Time_Clickstream_Analytics_with_Apache/links/6691464a3e0edb1e0fe0ce1c/Real-Time-Clickstream-Analytics-with-Apache.pdf

[4]Hanamanthrao, R., & Thejaswini, S. (2017). Real-time clickstream data analytics and visualization. 2017 2nd IEEE International Conference on Recent Trends in Electronics, Information & Communication Technology (RTEICT). https://doi.org/10.1109/rteict.2017.8256978

[5]Ehsan, A., Abuhaliqa, M. A. M. E., Catal, C., & Mishra, D. (2022). RESTful API Testing Methodologies: Rationale, Challenges, and Solution Directions. Applied Sciences, 12(9), 4369. https://doi.org/10.3390/app12094369

[6]Yusof, M. K., Man, M., & Ismail, A. (2022). Design and Implement of REST API for Data Integration. 2022 International Conference on Engineering and Emerging Technologies (ICEET). https://doi.org/10.1109/iceet56468.2022.10007414

[7]M. Gumber, A. Jain and A. L. Amutha, "Predicting Customer Behavior by Analyzing Clickstream Data," *2021 5th International Conference on Computer, Communication and Signal Processing (ICCCSP)*, Chennai, India, 2021, pp. 1-6, doi: 10.1109/ICCCSP52374.2021.9465526.

[8]Liu Y, Fan S, Xu S, Sajjanhar A, Yeom S, Wei Y. Predicting Student Performance Using Clickstream Data and Machine Learning. Education Sciences. 2023; 13(1):17. https://doi.org/10.3390/educsci13010017

[9]Ulitzsch, E., Ulitzsch, V., He, Q. et al. A machine learning-based procedure for leveraging clickstream data to investigate early predictability of failure on interactive tasks. Behav Res 55, 1392–1412 (2023). https://doi.org/10.3758/s13428-022-01844-1

[10]Severeyn, E., Alexandra La Cruz, Matute, R., & Estrada, J. (2023). Neural Networks for Customer Classification Through Clickstream Analysis. https://doi.org/10.1109/etcm58927.2023.10309081